Dự án nhỏ: kế hoạch kiểm thử và tự động hóa

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Kiểm thử phần mềm nhập môn.
- Chương 6: Chất lượng và thực hành tổng hợp.
- Các bài trong chương: Hiệu năng, khả dụng và tương thích; Dự án nhỏ: kế hoạch kiểm thử và tự động hóa.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Tạo gói kiểm thử có thể chạy lại cho một chức năng học trực tuyến.

Kiến thức nền

Tự động hóa phù hợp với ca lặp lại, kết quả mong đợi ổn định và giá trị phát hiện lỗi đủ lớn so với chi phí bảo trì. Không tự động hóa mọi thao tác chỉ vì có công cụ. Tách unit test nhanh, API/integration và một số luồng giao diện quan trọng theo nhu cầu sản phẩm.

Một test tốt có chuẩn bị, hành động và kiểm tra kết quả cụ thể. Các test nên độc lập, dữ liệu có thể tái tạo và dọn trong phạm vi đã tạo. Khi test chập chờn, tìm nguyên nhân như chờ bất đồng bộ hoặc phụ thuộc thứ tự, không đơn giản chạy lại tới khi xanh rồi bỏ qua.

Dự án chọn chức năng ghi danh hoặc nộp quiz: lập phạm vi, rủi ro, test case, dữ liệu, báo cáo lỗi và kết quả hồi quy. Phân biệt chức năng chưa kiểm tra với test đã qua. Tài liệu kết thúc phải nêu rủi ro còn lại; quiz không tự xác nhận bộ test thực tế đã được triển khai.

1. Phạm vi gói bàn giao

Chọn đăng ký khóa miễn phí làm tính năng mục tiêu. Tài liệu cần mục tiêu, phạm vi không làm, rủi ro, môi trường, dữ liệu, trách nhiệm, điều kiện bắt đầu/kết thúc và cách báo cáo. Chỉ tự động hóa các kiểm tra có kỳ vọng rõ, không tạo assertion theo dữ liệu đang trả về để test luôn xanh.

2. Ví dụ test độc lập không gọi hệ thống thật

```
def can_open_lesson(is_free, enrolled):
    return is_free or enrolled

def test_guest_cannot_open_restricted_lesson():
    assert can_open_lesson(False, False) is False
```

```
def test_enrolled_can_open_restricted_lesson():
    assert can_open_lesson(False, True) is True

def test_free_lesson_is_open():
    assert can_open_lesson(True, False) is True
```

Đây là mô hình luật đơn giản phục vụ học cách viết test, không phải toàn bộ hàm phân quyền của LMS. Trong dự án thật còn trạng thái khóa, tài khoản và vai trò; cần test implementation thật qua interface phù hợp.

3. Từ ví dụ tới kiểm tra tích hợp

Tạo user/course lab trong fixture, gọi API thực ở môi trường test, xác nhận phản hồi và dữ liệu. Mỗi test phải tự có điều kiện trước và dọn dữ liệu của nó. Test UI chờ điều kiện quan sát được thay vì sleep cố định dài. Khi flaky, thu trace và sửa nguyên nhân, không chỉ tăng timeout vô hạn.

4. Rubric tự đánh giá

Chấm yêu cầu có truy vết, bao phủ rủi ro, dữ liệu cô lập, assertions đúng, khả năng chạy lại và báo cáo trung thực. Nộp 12 test thiết kế, ba test tự động, kết quả thực chạy và hai báo cáo lỗi nếu tìm thấy; nếu dùng lỗi giả định phải gắn nhãn giả định. Không bịa lỗi hay kết quả chỉ để đủ số lượng bài nộp.

> **Bài tập vận dụng bổ sung**
>
> Nộp kế hoạch một trang, 12 test gồm quyền/biên/trạng thái/dữ liệu, hai báo cáo lỗi mẫu ghi rõ là tình huống giả định và ba test tự động có thể chạy lại trên lab. Không tạo lỗi hoặc kết quả giả vào hệ thống thật.

Hướng giải bài tập bổ sung

Đánh giá theo tiêu chí: truy vết yêu cầu, kết quả mong đợi rõ, dữ liệu độc lập, kiểm tra có ý nghĩa, bằng chứng chạy và giới hạn. Không tính số lượng test là thước đo duy nhất.

Câu hỏi tự kiểm tra

1. Ca nào thường phù hợp tự động hóa?
2. Ba phần cơ bản của test thường là gì?
3. Test độc lập có lợi ích gì?
4. Test chập chờn nên được xử lý thế nào?
5. Báo cáo kết thúc kiểm thử cần nêu gì?

Gợi ý đáp án và điểm cần nhớ

- Ca lặp lại có kỳ vọng ổn định và giá trị rõ. Cần cân nhắc giá trị và bảo trì.

- Chuẩn bị, hành động, kiểm tra. Test cần thiết lập trạng thái, thực hiện và xác minh.
- Giảm phụ thuộc thứ tự chạy. Có thể chạy lại riêng và theo thứ tự khác.
- Tìm nguyên nhân dữ liệu, thời gian hoặc phụ thuộc. Flaky test làm giảm độ tin cậy của phản hồi.
- Phạm vi, kết quả và rủi ro còn lại. Người đọc cần hiểu cả bằng chứng lẫn giới hạn.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf

Dự án nhỏ: kế hoạch kiểm thử và tự động hóa — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Bài thực hành: chọn ghi danh hoặc nộp quiz trong lab, tạo 12 ca có quyền/biên/trạng thái, ba test tự động có assertion và báo cáo kết quả. Tách dữ liệu chuẩn bị, hành động, xác minh và cleanup.

```
def test_valid_boundary():
    assert valid_score(0)
    assert valid_score(10)
```

Đoạn trên chạy sau định nghĩa valid_score của bài biên; không phải test kết nối vào hệ thống thật.

Hướng giải: test tự động phải phát hiện được hành vi sai có ý nghĩa, không chỉ khẳng định dữ liệu vừa tự tạo bằng đúng đoạn code tương tự. Ghi rõ test qua, fail, chưa chạy và giới hạn. Kết quả mẫu không phải bằng chứng tác vụ đó đã được chạy thật.

Nguồn học tiếp

https://docs.pytest.org/en/stable/how-to/assert.html